



Generalized preemption models for single-machine dynamic scheduling problems

FRANÇOIS M. JULIEN , MICHAEL J. MAGAZINE & NICHOLAS G. HALL

To cite this article: FRANÇOIS M. JULIEN , MICHAEL J. MAGAZINE & NICHOLAS G. HALL (1997) Generalized preemption models for single-machine dynamic scheduling problems, IIE TRANSACTIONS, 29:5, 359-372, DOI: [10.1080/07408179708966341](https://doi.org/10.1080/07408179708966341)

To link to this article: <https://doi.org/10.1080/07408179708966341>



Published online: 30 May 2007.



Submit your article to this journal [↗](#)



Article views: 57



View related articles [↗](#)



Citing articles: 1 View citing articles [↗](#)

Generalized preemption models for single-machine dynamic scheduling problems

FRANÇOIS M. JULIEN¹, MICHAEL J. MAGAZINE² and NICHOLAS G. HALL³

¹*Faculty of Administration, University of Ottawa, Ottawa, Ontario, Canada K1N 6N5*

²*Department of Quantitative Analysis and Operations Management, University of Cincinnati, Cincinnati, OH 45221*

³*Department of Management Sciences, Ohio State University, Columbus, OH, 43210-1399, USA*

Accepted May 1996

In the extensive scheduling literature, job preemption, if allowed, implies that the processing of a partly completed job is temporarily halted and later resumed at the same point. However, little attention has been given to problems where job preemption is allowed under the condition that either some startup time delay must be incurred or some fraction of work must be repeated if preemption occurs. We generalize the notion of job preemption by using models representing these conditions. The models are applied to studying the dynamic single-machine scheduling problems of minimizing total flow time, and of minimizing maximum lateness, subject to arbitrary and unknown job ready dates. On-line optimal dispatching rules, which consider only available – as opposed to look-ahead – information, are developed. These rules determine, on arrival or completion of each job, which available job should next be processed by the machine. A special case of our models, the preempt-repeat scenario, where preempted jobs must be totally repeated, is suggested as heuristic for the equivalent non-preemptive static problem where all ready dates are known and given. A computational study is performed to determine the potential benefits of reducing startup time delays or work repetition fractions in the context of continuous improvement of manufacturing systems.

1. Introduction

A considerable volume of literature has addressed non-preemptive scheduling problems with arbitrary job ready dates. A non-preemptive problem is one in which each job, once started, must be completed before another job can be started on the machine. Such an environment is common to many industrial settings. In a non-preemptive static scheduling problem, all job parameters, including processing times, due dates and ready times are known, fixed, and given. Most of these problems are notoriously difficult and some general purpose technique such as branch-and-bound or dynamic programming is often necessary to find the optimal schedule by using the available job data. We say that the scheduling decisions taken for static problems are made off-line, since data for all jobs, even those yet to arrive, are taken into account before decisions are made.

In some cases, the optimal solution of a non-preemptive static problem might require the use of inserted idle time on the machine: this implies scheduling some idle time on the machine when one (or more) job is ready for processing, so that a more ‘important’ job, which will become available in the near future, can be processed as soon as it arrives. Inserting idle time might be considered only if information about the nature of jobs and their

ready times is available. Moreover, it might be avoided in static preemptive scheduling problems.

Preemptive scheduling problems are those in which processing on a job can be temporarily interrupted, and resumed later from the point at which preemption occurred. Solutions to such problems, which will hereafter be referred to as preempt-resume problems, have been most useful in providing lower bounds on the solution value for the corresponding non-preemptive problem. This is because preempt-resume problems are usually much easier to solve: typically, only the characteristics of jobs known to be in the system at a given point in time are needed to make an optimal scheduling decision, even if look-ahead information is also available. Scheduling decisions are made on-line if, at the time of the decision, only the information related to available jobs is used (i.e., without the use of look-ahead information).

In dynamic problems, job parameters might not be known until the arrival of the job, so that, at any point in time, scheduling decisions must be made without the type of look-ahead information about jobs yet to arrive that is available in static problems. When ready times are not known with certainty, and other job characteristics become known only when the job arrives in the system, scheduling problems define an important need: the ability

to reschedule dynamically. Baker [1] defines *dispatching* as an on-line decision-making structure for dynamic problems where at each decision point (i.e., job completion time or ready time), the necessary information is obtained only from the current set of available jobs, excluding predictions of future events. Scheduling decisions can be made at chronologically ordered points in time, based on currently known information.

Allowing job preemption provides additional dispatching flexibility, enabling schedulers to make appropriate changes to the schedule and to interrupt a job if a more urgent one arrives. However, it is reasonable to assume that in some cases, preempted jobs cannot simply be resumed, but that a certain delay or startup time must be incurred before jobs can be resumed. This delay could take the two following forms. First, it might be a setup time, needed to change the tooling back to the specifications of the preempted job, as in the microelectronics industry, where each job is a batch of items of a given type requiring a specific machine setup. Secondly, it might be that some proportion, or perhaps all, of the processing needs to be repeated. For example, the compilation and execution of a computer program might need to be repeated if preempted. Finally, the time needed by an operator to 'get back into the job', i.e., to remember where he/she had left the work, what he/she had done, and to plan what still needs to be done, can be modelled. In pick-up and delivery services, for example, if the dispatcher asks a truck to interrupt momentarily its route to take care of a new customer, some time delay might result, associated with the driver's returning to the regular route after the interruption. This paper addresses dispatching rules for single-machine dynamic scheduling problems given this more general definition of job preemption, which considers startup times, for the two cases outlined above. These models will be applied to two well-known single-machine dynamic scheduling problems: minimizing total flow time, and minimizing maximum lateness.

These two problems have three characteristics in common. First, the static non-preemptive versions of these problems, with arbitrary ready dates, are known to be NP-hard [2]. Secondly, the static versions of these problems with no ready times (i.e., where all jobs become available simultaneously), are easily solvable: the shortest processing time sequencing rule (SPT), proposed by Smith [3] for the total flow time problem, and the earliest due date sequencing rule (EDD) studied by Jackson [4] for the maximum lateness problem, are still two of the most important and fundamental results in the scheduling literature. Thirdly, the preempt-resume static versions of these problems are also tractable. For the total flow time problem, it is sufficient to choose, at each job arrival or completion point, the job with the shortest remaining processing time (SRPT) to receive service next. For the maximum lateness problem, however, an optimal schedule

is such that, at any point of time, the machine is always processing the available job with earliest due date [1].

For these two problems, and any other that would have these three characteristics, allowing preemption in a 'resume' mode has the significant theoretical impact of changing computationally hard problems (static, non-preemptive version) into polynomially solvable problems (static, preempt-resume case). Scheduling decisions made in polynomial time on-line, i.e., without look-ahead information, are optimal for the static problem in the preempt-resume mode: in fact, under this mode, look-ahead information is not even needed and does not have to be available to make globally optimal decisions for problems having the three above-mentioned characteristics. Hence the dynamic version of these problems may be solved optimally.

From a practical point of view, on-line dispatching is easier to perform and even more appealing if preemption is possible and allowed, as schedulers need not worry about the counter-intuitive decision of inserting idle time when jobs are in queue for service. A preempt-resume environment, achievable through elimination of delays associated with preemption, such as investment in setup time reduction, is certainly desirable. However, as preemption delays will persist until this goal is achieved, we develop dispatching rules for the more general preemption scenarios of non-zero setup times and partial repetition of preempted jobs.

In Section 2, we present a brief literature review along with the notation, definitions and assumptions of the two new preemption models. Section 3 deals with the two problems of minimizing total flow time, and total weighted flow time. Although the static preempt-resume version of this latter problem with arbitrary ready times is NP-hard, its analysis represents a simple generalization of the unweighted problem; the weighted shortest remaining processing time rule (WSRPT), which is optimal for the static problem with no ready times, is a reasonable and effective heuristic for the preempt-resume problem with ready times. Simple dispatching rules used to solve the dynamic scheduling preemption problems are developed. A 'preempt-repeat' dispatching procedure, where preempted jobs must be totally repeated, is proposed as a reasonable heuristic for the NP-hard static non-preemptive problems, and special cases for which the algorithm is optimal for this problem are identified. Worst-case analysis is performed for the proposed heuristic. Section 4 follows essentially the same outline for the maximum lateness problem. The focus of Section 5 is to illustrate the potential advantages of continuous manufacturing improvement through reduction of the time delays associated with preemption: the goal is achieving perfect dispatching flexibility attainable in the preempt-resume mode. We conclude by summarizing the main results of this paper and suggesting areas for future research.

2. Literature review and preemption-startup models

Let p_j, r_j, w_j and d_j be the processing time, ready time, weight and due date of job j , $j = 1, 2, \dots, N$ respectively, where N represents the total number of jobs for the static problems. For the minimization of total weighted flow time of the N jobs on a single machine, if $r_j = 0 \forall j$, the WSPT rule proposed by Smith [3] which sequences the jobs in non-decreasing order of $\rho_j = p_j/w_j$, is optimal. For arbitrary r_j , the problem becomes NP-hard, even when $w_j = 1 \forall j$, as shown by Lenstra *et al.* [2]. Numerous branch-and-bound algorithms have been suggested for both unweighted and weighted problems: see [5–9] and, for a related problem, [10]. The heuristics in these papers are all dispatching rules in which preemption or inserted idle time are not permitted. Finally, Posner [11] has developed a polynomial time algorithm for the static non-preemptive problem, assuming that jobs may be clustered (based on their ready time), that can be used heuristically for the problem without clusters.

For the problem of minimizing maximum lateness, if $r_j = 0 \forall j$, the EDD rule proposed by Jackson [4], which schedules jobs in non-decreasing order of d_j , is optimal. Again, for arbitrary ready times, the problem is NP-hard, as shown by Lenstra *et al.* [2].

Our goal is to develop dispatching rules for these problems that are *on-line optimal*, that is, dispatching rules such that at every decision point, the action chosen minimizes the objective in a myopic sense, i.e., assuming that no other job would enter the system before all the currently available jobs are completed. On-line optimality is a sensible criterion because, in an on-line environment, the ready times of future jobs are not always known with certainty. Not unexpectedly, on-line optimality does not imply *off-line optimality*, which pertains to solutions that are optimal if all job characteristics are initially known, as will be illustrated later. At the completion of a job, a decision must be taken. Inserting idle time for a single machine problem is reasonable only if ready times of future jobs are known. When ready times are not known, decision makers will typically keep the machine busy as long as there are jobs available, because it might be possible to complete one or more of the currently available jobs before the arrival of the next one. However, if an available job is started, and no preemption is allowed, one takes the risk of having a relatively more important job arriving shortly thereafter, which might then have to wait a long time before being processed. Having preemption as an option at job arrival times, even in a ‘non-preempt-resume’ mode, provides additional flexibility in a dynamic environment. Thus on-line optimal decision rules are needed for two types of decision point: job ready times and completion times.

As mentioned in the introduction, a certain time penalty might be incurred if preemption is allowed. We present and solve (using the criterion of on-line opti-

mality) two preemption models, the preempt-setup and preempt-startup models, to capture such time delays.

Let $(p_j - v)$ be the *remaining processing time* of job j if it is preempted after v units of time: when j returns to the machine, it will require $(p_j - v)$ time units plus a certain preemption time ‘penalty’ or ‘delay’ to be completed. Let $t_j \geq (p_j - v)$, the *effective processing time* of j , be the time needed by the machine to complete j assuming that there are no more preemptions of j .

For the *Preempt-setup* model, we assume that each time that job j is started on the machine, whether initially or after having been preempted, a setup s_j must take place. (The setup is also subject to preemption.) Thus the initial effective processing time of job j is $t_j = s_j + p_j$. Such a situation could arise in several settings: for example, in the microchip industry, job j might be some batch of wafers, which could be preempted and resumed, subject to the setup time. Thus if j gets interrupted after v units of time, then, in general, the (new) effective processing time of j will be: $t_j \leftarrow (t_j - \gamma)$, where $\gamma = \max\{0, v - s_j\}$. Thus this model assumes preempt-resume operation for the processing time, with preempt-repeat for the setup [12].

For the *Preempt-Startup* model, the initial effective processing time will simply be $t_j = p_j$. If j is preempted after v units of time, let $t_j \leftarrow (t_j - v) + \alpha_j v = t_j - (1 - \alpha_j)v$ be the (new) effective processing time, where we will assume that $0 \leq \alpha_j \leq 1$. Thus the new effective processing time will be the previous effective processing time reduced by a fraction of the work done during the last visit at the machine. α_j may be used to model the startup time taken by the operator to release the job, or it may represent that some proportion α_j (but not necessarily all) of the processing must be repeated. For example, if the machine is an oven, some if not all of the heating of the job might need to be repeated if the job is preempted. Compilation and execution of computer programs might also need to be repeated if preempted. Although in this paper we assume that α_j is given and fixed for each job, this parameter can also be a function of the amount of processing that took place before interruption.

Figs 1 and 2 illustrate the difference between the effective processing time in both models. The curves represent the effective processing time of a job j given that it was preempted at some time $t \leq t_j$. We now study the two models in parallel for the dynamic scheduling single-machine problems of minimizing total flow time and minimizing total weighted flow time.

3. Minimizing total flow time and total weighted flow time

3.1. On-line optimal dispatching rules for both preemption models

Consider the dynamic version of these problems in which no information is known about a job until it arrives, at

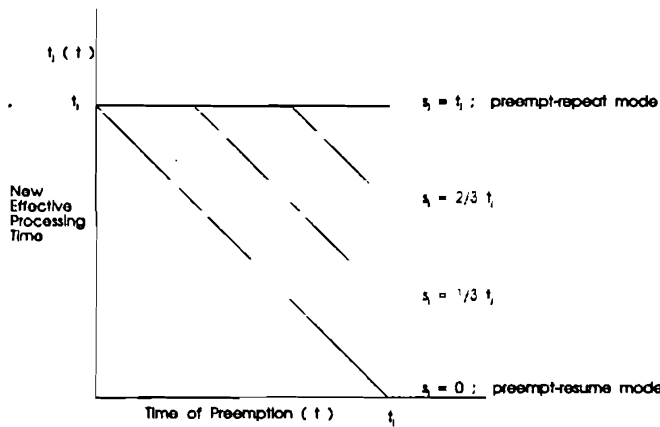


Fig. 1. Effective processing time for the preempt-setup model given preemption at time t for four values of s_j .

which point all relevant information becomes known. Because the total flow time problem is a special case of the total weighted flow time problem with $w_j = 1 \forall j$, we present our analysis in terms of the weighted problem, for brevity of the exposition. We now characterize an on-line optimal policy for this problem.

Observation 3.1. *At the beginning of the schedule, and at every job completion time, the weighted shortest effective processing time rule (WSEPT) is on-line optimal: sequence the available jobs in non-decreasing order of $\rho_j = t_j/w_j$.*

Because nothing is known about the characteristics of jobs that have not yet arrived, it is clearly suboptimal to allow idle time if a job is available. Applying the WSEPT rule, which is equivalent to the WSRPT rule for the problem with no delay penalty associated with preemption, minimizes the objective for the set of available jobs.

Let n denote the number of jobs in the system when a new job, $(n+1)$, arrives, with job 1 on the machine, and let 0 be the start time of job 1: thus $0 < r_{n+1} < t_1$. An on-

line optimal dispatching rule at time r_{n+1} will produce a schedule that minimizes the total weighted flow time of the $n+1$ jobs, as though no other job would enter the system before all currently available jobs were completed. The on-line optimal procedure must compare two possible options: preemption or non-preemption of job 1. If non-preemption is chosen, job 1 remains on the machine, and jobs $2, 3, \dots, n+1$ are sequenced according to WSEPT, by Observation 3.1. However, if preemption is chosen, the on-line optimal schedule for the $n+1$ jobs will be obtained by applying WSEPT once t_1 has been adjusted. This procedure will be repeated at every new job arrival time. Hence, the algorithm implies that, unlike in the well-known preempt-resume model, if preemption is preferred, then the arriving job, $(n+1)$, will not necessarily replace job 1 on the machine. Numerical examples, summarized in Table 1 and Fig. 3, illustrate this.

The preempt-setup instance in Table 1 represents the case where both jobs 2 and 3 arrive after s_1 is completed: similar examples can be constructed where both jobs 2 and 3 arrive before s_1 is completed or only job 2 arrives before s_1 is completed. Both the preempt-setup and the preempt-startup instances illustrate a situation shown in Fig. 3, where job 1 is not preempted when job 2 arrives, is preempted when job 3 arrives, and is replaced on the machine by job 2 at time r_3 .

Let $\underline{1}$ be the part of job 1 completed before preemption, $\underline{1}'$ the remaining part of job 1 and $F(\sigma)$ the total flow time of schedule σ . When job 2 arrives, the on-line optimal choice between σ_1 and σ_2 (see Fig. 3) is σ_1 , or no preemption, in both examples. Note that in a dynamic setting, it is not known that job 3 will be arriving shortly after job 2, so that σ_1 and σ_2 are the only reasonable options at the arrival of job 2. As job 1 was not preempted, when job 3 arrives, the on-line choice now is between schedules σ_3 and σ_4 : σ_4 is on-line optimal. The optimal (using look-ahead information) schedule, σ^* , is also given.

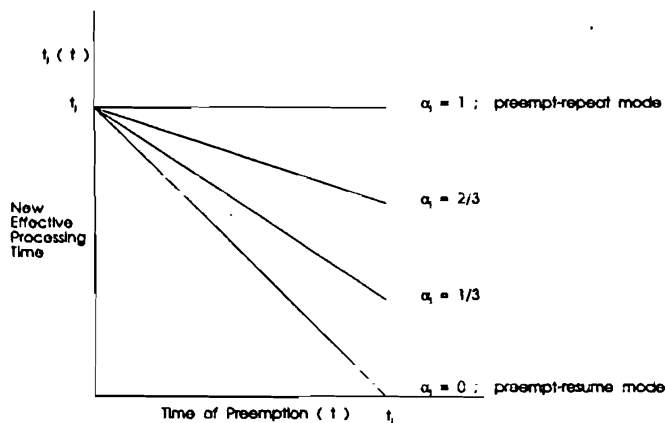
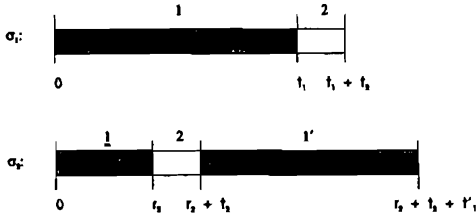


Fig. 2. Effective processing time for the preempt-startup model given preemption at time t for four values of α_j .

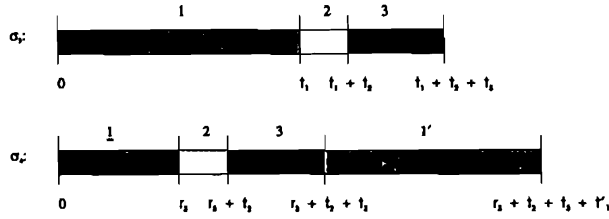
Table 1. Numerical examples for preempt-setup and preempt-startup models

Example for preempt-setup model										
Ex. 1:	j	s_j	p_j	r_j	w_j	$F(\sigma_1)$	$F(\sigma_2)$	$F(\sigma_3)$	$F(\sigma_4)$	$F(\sigma^*)$
	1	4	6	0	1	17.1	18	27	26.2	25.8
	2	1	1	4.9	1					
	3	1	2	5.1	1					
Example for preempt-startup model: $\alpha = 0.5$										
Ex. 2:	j	p_j	r_j	w_j		$F(\sigma_1)$	$F(\sigma_2)$	$F(\sigma_3)$	$F(\sigma_4)$	$F(\sigma^*)$
	1	10	0	1		16.6	16.7	26	25	24.5
	2	2	5.4	1						
	3	3	5.6	1						

a) Arrival of job 2:



b) Arrival of job 3:



c) Optimal schedule:

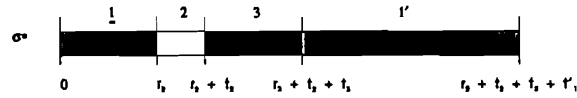


Fig. 3. Each time a new job arrives, two schedules are compared; non-preemption of the job on the machine, or preemption of that job.

Although $t_2 < t_3$ in these examples, job 2 did not force preemption of job 1 while job 3 did. Preemption of job 1 at time r_3 would remain on-line optimal if $4 \leq r_2 \leq r_3 \leq 5.5$ in the preempt-setup example and if $16/3 \leq r_2 \leq r_3 \leq 6$ in the preempt-startup example. The optimal schedule for the static non-preemptive problem is given by σ^* for example 1, where we replace $\underline{1}$ by some idle time, and by σ_3 for example 2: the total flow times are 26.7 for the preempt-setup instance and 26 for the preempt-startup example. These total flow times are higher than those of σ_4 , the on-line optimal schedule, in both instances: if preemption is permitted, even under a preemption time delay, starting job 1 is preferable to inserting idle time as some of the processing of job 1 gets completed before job 2 arrives.

Let $[j]$ be the j th job in the WSEPT sequence, $j \leq n \leq N$, and $[k]$ be the job on the machine when job $(n+1)$ arrives. Assume $[k]$ was started at time $X_{[k]}$ so that $(n+1)$ arrived $r_{n+1} - X_{[k]}$ units of time after the start of the current processing of $[k]$. Let $\delta = \min\{r_{n+1} - X_{[k]}, s_{[k]}\}$, and let $t'_{[k]} = (t_{[k]} - r_{n+1} + X_{[k]} + \delta)$ be the new effective processing time of $[k]$ if preempted at time r_{n+1} , $\rho'_{[k]} = t'_{[k]}/w_{[k]}$, and B be the subset of jobs $j \in \{1, 2, \dots, n+1\}$ such that $\rho_j < \rho'_{[k]}$. Finally, let $F_w(\sigma)$ be the total weighted flow time

of a schedule σ . We have the following results for the preempt-setup model.

Lemma 3.2. *If $\rho_{n+1} \geq \rho'_{[k]}$, there exists an on-line optimal schedule where $[k]$ is not preempted.*

Proof: Note that jobs $[1], \dots, [k-1]$ must have arrived after $[k]$ started. Let $\rho_{[j]} < \rho'_{[k]} \leq \rho_{[j+1]}$, so that we would insert $[k]'$ between $[j]$ and $[j+1]$ in the WSEPT sequence. If $\rho_{n+1} \geq \rho'_{[k]}$ then if preemption occurs, job $(n+1)$ would be scheduled after $[k]'$, which denotes the unprocessed portion of $[k]$. Consider two schedules:

$$\sigma: ([k], [1], \dots, [j], [j+1], \dots, (n+1), \dots, [n])$$

non-preemption;

$$\sigma': ([k], [1], \dots, [j], [k]', [j+1], \dots, (n+1), \dots, [n])$$

preemption.

If $k = 1$, then σ is preempt-resume optimal and hence preempt-setup on-line optimal. Suppose $k > 1$; let τ and τ' represent the schedules obtained by removing job $(n+1)$ from σ and σ' , respectively. Now $F_w(\tau) \leq F_w(\tau')$ because otherwise $[k]$ would have been preempted by one of the jobs in B at an earlier instance. Let F_i and F'_i be the flow times of job i in σ and σ' respectively, then $F_{n+1} \leq F'_{n+1} = F_{n+1} + \delta$, so that $w_{n+1}F_{n+1} \leq w_{n+1}F'_{n+1}$. Finally, the contribution of job $(n+1)$ to the 'costs' $w_i F_i$ and $w_i F'_i$ is identical, for all jobs i scheduled after $(n+1)$. Therefore $F_w(\sigma) \leq F_w(\sigma')$ and no preemption should occur. ■

From this result we conclude that it is not necessary to preempt job $[k]$ in favor of job $[1]$ at the arrival of job $(n+1)$ unless

$$\rho_{n+1} < \rho'_{[k]}. \quad (1)$$

Theorem 3.3. *Assuming (1) holds, then, for the preempt-setup model, it is on-line optimal to preempt $[k]$ in favor of job $[1]$ at the arrival of job $(n+1)$ if and only if:*

$$r_{n+1} - X_{[k]} \leq \left(t_{[k]} \sum_{i \in B} w_i - w_{[k]} \sum_{i \in B} t_i - \delta \sum_{i \notin B} w_i \right) / \sum_{i \in B} w_i. \quad (2)$$

Proof: Suppose (1) holds: hence $(n+1) \in B$ and will be scheduled before $[k]'$ if preemption is preferred. As before, let $\rho_{[j]} < \rho'_{[k]} \leq \rho_{[j+1]}$. Consider the following schedules.

$$\sigma: ([k], [1], \dots, [j], [j+1], \dots, [n])$$

non-preemption;

$$\sigma': ([k], [1], \dots, [j], [k]', [j+1], \dots, [n])$$

preemption.

Let $F_w(\sigma') = F_w(\sigma) + \beta_1 - \beta_2$, where β_1 and β_2 are respectively the increase and decrease in total weighted completion time resulting in preempting job $[k]$ at the arrival of job $(n+1)$; then

$$\beta_1 = \delta \left(\sum_{i \notin B} w_i \right) + w_{[k]} \left(\sum_{i \in B} t_i \right);$$

$$\beta_2 = (X_{[k]} + t_{[k]} - r_{n+1}) \sum_{i \in B} w_i.$$

Preemption is on-line optimal if $\beta_2 - \beta_1 \geq 0$, i.e., if and only if condition (2) is met. ■

Parallel results hold for the preempt-startup model. Define $t'_{[k]} = t_{[k]} - (1 - \alpha_{[k]})(r_{n+1} - X_{[k]})$. Clearly, Lemma 3.2 holds for the preempt-startup model. Furthermore, Theorem 3.4 can also be shown to hold. Because the proof of the following result closely parallels that of Theorem 3.3, we have not included it, for brevity.

Theorem 3.4. *Assuming (1) holds, then, for the preempt-startup model, it is on-line optimal to preempt $[k]$ if and only if the following condition holds:*

$$r_{n+1} - X_{[k]} \leq \left(t_{[k]} \sum_{i \in B} w_i - w_{[k]} \sum_{i \in B} t_i \right) / \left(\sum_{i \in B} w_i + \alpha_{[k]} \sum_{i \notin B} w_i \right). \quad (3)$$

These results follow our intuition: for preemption to occur, we require that job $(n+1)$ be more 'urgent' than what remains of job $[k]$, and that it arrives early enough that the reduction in weighted flow times of the relatively 'important' jobs $j \in B$ exceeds the weighted delay imposed on less 'important' jobs $j \notin B$. For the unweighted problem, letting $b \leq (N+1)$ be the cardinality of set B , the conditions stated in Theorems 3.3 and 3.4 become:

$$r_{n+1} - X_{[k]} \leq \left[bt_{[k]} - \sum_{i \in B} t_i - \delta(n+1-b) \right] / b$$

(preempt – setup);

$$r_{n+1} - X_{[k]} \leq \left(bt_{[k]} - \sum_{i \in B} t_i \right) / [b + \alpha_{[k]}(n+1-b)]$$

(preempt – startup);

In view of these results, we propose the following on-line optimal dispatching algorithm. Let X_j be the start time of job j on the machine. Note that j may be preempted, so X_j will also be used to represent the most recent start time of j .

On-line dispatching preempt-setup/preempt-startup algorithm:

- Step 1.** At the start of the schedule, and at every completion time, C_i , of job i , start the job j with smallest ρ_j among all available jobs; let $X_j \leftarrow C_i$ (or $X_j = 0$ if j is the first job scheduled).
- Step 2.** At every job arrival, if $[k]$ and $(n+1)$ are respectively the job being processed and the job

arriving, if $\rho_{n+1} < \rho'_{[k]}$ and condition (2) for the preempt-setup model, or (3) for the preempt-startup model, holds, preempt $[k]$ and update $t_{[k]}$ as follows:

$$t_{[k]} \leftarrow t_{[k]} - \max\{0, r_{n+1} - X_{[k]} - s_{[k]}\} \quad (4)$$

preempt – setup model;

$$t_{[k]} \leftarrow t_{[k]} - (1 - \alpha_{[k]})(r_{n+1} - X_{[k]}) \quad (5)$$

preempt – startup model.

Start job j with smallest ρ_j among all available jobs. If $\rho_{n+1} \geq \rho'_{[k]}$ or condition (2) or condition (3) does not hold, do not preempt $[k]$.

Step 3. Repeat steps 1 and 2 until all N jobs have been processed to completion. Stop.

This algorithm is straightforward and easy to implement. The computational effort is determined by the $O(N)$ evaluations of (2) or (3) at step 2, where each such evaluation can be performed in $O(N)$ time. The algorithm thus runs in $O(N^2)$ time.

As illustrated in Figs. 1 and 2, both preemption models bridge a gap between two extreme cases. If $s_j = 0 \forall j$ in the preempt-setup model, or $\alpha = 0$ in the preempt-startup model, we obtain the preempt-resume dispatching mode, whereas if $p_j/s_j \rightarrow 0$ in preempt-setup so that $t_j \rightarrow s_j$ and, at the limit, $\delta = r_{n+1} - X_{[k]}$, or $\alpha = 1$ in preempt-startup, we obtain the preempt-repeat dispatching mode.

Clearly, given a set of N jobs, a solution to a dynamic preempt-repeat instance is feasible for the corresponding static non-preemptive problem with arbitrary ready times, as preemption becomes equivalent to inserting idle time. Moreover, for the preempt-repeat model, conditions (2) and (3) converge to the following (weighted and unweighted case respectively):

$$r_{n+1} - X_{[k]} \leq \left(t_{[k]} \sum_{i \in B} w_i - w_{[k]} \sum_{i \in B} t_i \right) / \sum_{i=1}^{n+1} w_i$$

$$r_{n+1} - X_{[k]} \leq \left(bt_{[k]} - \sum_{i \in B} t_i \right) / (n+1). \quad (6)$$

Thus, an on-line dispatching preempt-repeat algorithm is obtained from the above-outlined algorithm by replacing conditions (2) or (3) by condition (6), and by deleting (4) and (5) because in this environment t_j remains unchanged if j is preempted. This preempt-repeat algorithm can thus be used heuristically for the static problem, where preemption corresponds to inserting idle time. In the following subsection we present conditions under which this on-line dispatching preempt-repeat algorithm solves the static non-preemptive problem.

3.2. On-line preempt-repeat dispatching for the static non-preemptive problem

Although the preempt-repeat rule is never beneficial for the static non-preemptive problem as inserted idle-time

gives the same flow benefit without unnecessary machine usage [1], our proposed preempt-repeat heuristic is presented as an alternative to another well-known 'non-look-ahead' (on-line) heuristic, which was first suggested by Chandra [6] for the non-weighted case and adapted by Hariri and Potts [9] for the weighted problem, which does not allow job preemption. The C/HP heuristic can be described as follows: at the beginning of the schedule and at every job completion time, start the available job with smallest ρ_j .

Hariri and Potts [9] provide sufficient conditions for C/HP to be optimal for the static problem. Let job i be the i th job in the schedule obtained with C/HP. The completion times of the N jobs are thus $C_1^* = r_1 + p_1$, and for $i = 2, 3, \dots, N$, $C_i^* = \max \{r_i, C_{i-1}^*\} + p_i$. Job v_j is said to be last in a block S_j if $C_{v_j}^* \leq r_i$, for $i = v_j + 1, \dots, N$. A set of jobs $S_j = \{u_j, u_j + 1, \dots, v_j\}$ forms a block if the following conditions are met:

- (a) $u_j = 1$, or $u_j - 1 = v_{j-1}$ is the last job in block S_{j-1} ;
- (b) job $i \in S_j$ is not the last job in block S_{j-1} ;
- (c) job v_j is the last job of block S_j .

For C/HP, u_j has the property that $r_{u_j} \leq r_i$ for $i = u_j + 1, \dots, N$. Given these definitions, sufficient conditions for the sequence $(1, 2, \dots, N)$ generated by C/HP to be optimal for the static non-preemptive problem are that the jobs within each block S_j are sequenced in non-decreasing order of ρ_j . We thus have the following result.

Lemma 3.5. *Let $(1, 2, \dots, N)$ be the sequence of jobs generated by the preempt-repeat algorithm, which can be partitioned into blocks of jobs S_j satisfying conditions (a), (b), and (c) above. This sequence is optimal for the static non-preemptive problem if jobs within each block S_j are sequenced in non-decreasing order of ρ_j .*

Proof: Under those conditions, the C/HP algorithm would be optimal for the static non-preemptive problem, and no preemption would occur under the preempt-repeat algorithm, so that both algorithms would yield the same schedule. ■

We now describe additional conditions under which the preempt-repeat algorithm is optimal for the static non-preemptive problem. The result is more general than that of Hariri and Potts [9], in that the resulting schedule can contain preemptions. The conditions that we use are similar to those described by Posner [11]; however, neither set of conditions implies the other.

We assume that all the jobs are indexed so that $r_i \leq r_{i+1}$, $i \leq N - 1$. Given a schedule obtained by the preempt-repeat algorithm, we define Isolated Clusters, IC, as subsets of the N jobs: let $IC_j = \{i_{j-1} + 1, \dots, i_j\}$, where $i_0 + 1$ is job 1, and

$$C1: r_k < r_{[1]_j} + p_{[1]_j} \quad \forall k \in IC_j;$$

$$C2: r_{[1]_j} < r_k + p_k \quad \forall k \in E_j;$$

$$C3: r_t \geq r_{[1]_j} + \sum_{k \in IC_j} p_k \quad \forall t > i_j;$$

where $[k]_j \in IC_j$ is the k th job in the WSPT sequence in IC_j , $1 \leq k \leq (i_j - i_{j-1} + 1)$ for each Isolated Cluster j , $E_j = \{k | r_k < r_{[1]_j}, k \in IC_j\}$, and with $|E_j| = c_j$ the number of jobs in IC_j that have arrived before $[1]_j$. Let $< k >_j \in E_j$ be the k th job in the WSPT sequence, $1 \leq k \leq c_j - 1$, within E_j .

Theorem 3.6. *The preempt-repeat algorithm solves the static non-preemptive problem of minimizing total weighted flow time if, in the sequence generated, the N jobs can be partitioned into z Isolated Clusters such that*

$$C4: \bigcup_{j=1}^z IC_j = \{1, \dots, N\};$$

$$C5: \text{For every IC, if } < k > \in E_j \\ \text{is being processed at time } r_{[1]_j},$$

$$r_{[1]_j} - r_{< k >} \leq \frac{p_{< k >} \left(\sum_{i=1}^{k-1} w_{< i >} + w_{[1]_j} \right) - w_{< k >} \left(\sum_{i=1}^{k-1} p_{< i >} + p_{[1]_j} \right)}{\sum_{i \in E_j} w_i + w_{[1]_j}}.$$

Proof: Given the definition of Isolated Clusters and C5, the preempt-repeat algorithm will schedule the jobs in each cluster j as follows. No job can complete processing without being preempted by $[1]_j$, from C2 and C5. Therefore, whatever scheduling of jobs $h \in E_j$ occurs before the arrival of $[1]_j$ will not affect the cost of the schedule. Note that by definition and from Lemma 3.2, no other job will preempt $[1]_j$. From C1, by the time $[1]_j$ completes processing, all the other jobs in the cluster are available. Those jobs are therefore completed in SWPT order. Finally, from C3, no other job becomes available until all jobs in the cluster are completed. The final sub-schedule for IC_j , ignoring any work later repeated, is $[1]_j, [2]_j, \dots, [i_j - i_{j-1} + 1]_j$.

To show that this schedule is optimal, we note that condition C5 makes it on-line optimal to preempt any job in favor of $[1]_j$. Because no other job within the cluster has smaller ρ , it is on-line optimal to finish the scheduling of $[1]_j$. Given C1, the scheduling of the remaining jobs in the cluster then becomes a trivial problem, provided that later jobs do not arrive until the entire cluster is scheduled. This last condition is provided by C3. It is clear that the schedule is optimal within any cluster: thus, as long as C4 holds, so does the result. ■

3.3. Worst-case analysis of the preempt-repeat heuristic

We conclude this section by providing examples where C/HP and the preempt-repeat algorithm perform poorly. We first show that the ratio of $F_w(\text{C/HP})$ over $F_w(\text{optimal})$ can be made arbitrarily large, even for the unweighted case.

Consider the following example.

$$\begin{array}{cccccc} i: & 1 & 2 & 3 & \dots & N \\ p_i: & M & \varepsilon & \varepsilon & \dots & \varepsilon \\ r_i: & 0 & \varepsilon & \varepsilon & \dots & \varepsilon \end{array}$$

For this instance, the C/HP schedule is $(1, 2, \dots, N)$, whereas, assuming that $M \geq (2 + 1/(N-1))\varepsilon$, the optimal schedule (given by the preempt-repeat algorithm), is $(1, 2, 3, \dots, N, 1)$, so that

$$\frac{F(\text{C/HP})}{F(\text{opt})} = \frac{NM + \frac{1}{2}(N^2 - 3N + 2)\varepsilon}{\frac{1}{2}(N^2 + N)\varepsilon + M}.$$

As $\varepsilon \rightarrow 0$, this ratio tends to N , the number of jobs.

In the following example, we show that the ratio $F(\text{preempt-repeat})$ over $F(\text{opt})$ can also be made arbitrarily large.

$$\begin{array}{ccccccccc} i: & 1 & 2 & 3 & 4 & \dots & N \\ p_i: & M & 1 & 1 & 1 & \dots & 1 \\ r_i: & 0 & \frac{1}{2}M - 1 & M - 1 & \frac{3}{2}M - 1 & \dots & \frac{1}{2}(N-1)M - 1 \end{array}$$

If $M \geq 6$ and $N \geq 3$, the optimal static non-preemptive schedule (obtained by C/HP) is $(1, 2, \dots, N)$. The preempt-repeat algorithm would start job 1, preempt it when job 2 arrives, return to job 1, preempt it again when 3 arrives, and so on: job 1 is continually preempted, and completed only after job N . The preempt-repeat schedule is $(1, 2, 1, 3, 1, \dots, 1, N, 1)$. Hence we have the following ratio, which tends to $\frac{1}{2}(M+2)$ for a fixed $M \geq 6$, as N tends to infinity:

$$\frac{F(\text{preempt-repeat})}{F(\text{opt})} = \frac{\frac{1}{2}(N(M+2) + M) - 1}{\frac{3}{2}M + N + 2}.$$

We have seen that conditions exist under which the C/HP and preempt-repeat heuristics are optimal for the static non-preemptive problem. The above examples show that both heuristics can perform poorly for that problem. These examples suggest that failing to preempt may cause C/HP to perform poorly, whereas greedy preemption is not always desirable in the preempt-repeat algorithm. This suggests that these algorithms might complement each other nicely, so that one potentially successful heuristic for the non-preemptive static problem could be to obtain a solution to the problem with each procedure and select the better of those two solutions.

We verified this hypothesis numerically by testing our preempt-repeat algorithm, C/HP and the combined algorithm on the total flow time problem against the lower bound obtained by the preempt-resume algorithm. Processing times and ready times were generated from a discrete uniform distribution over the integers $1, \dots, 50$ and $1, \dots, 2000$, respectively. We let $N = 20, 40, 60, 80$ and 100 to simulate different shop utilization rates, from very light traffic to very heavy traffic. We randomly generated 50 problems for each value of N . Table 2 gives the average percentage heuristic error above the lower

bound and how many times each heuristic gave the best solution for the combined algorithm.

Table 2 suggests that the preempt-repeat and C/HP heuristics complement each other nicely. When traffic is low to medium (i.e., $N \leq 60$ arrivals), preemption seems more desirable: at the arrival of each job, it is reasonable to assume that no other job will arrive soon, so that the greedy on-line optimality criterion should provide good global results: the preempt-repeat algorithm takes advantage of this. For high traffic (for $N \geq 80$, the expected inter-arrival time is smaller than the expected processing time of 25.5), preemption is much less desirable because the preemption time delay seems to adversely affect the flow time of many jobs already in queue or arriving shortly. The experiment suggests that it is worthwhile to take advantage of preemption in light to medium traffic, even if the preemption time delay is large. More benefits can be attained if the preemption time delay is reduced. This will be discussed further in Section 5.

4. Minimizing maximum lateness

In this section we provide an analysis similar to that of Section 3, for the dynamic problem of minimizing maximum lateness. If d_j is the due date of job j , and C_j the completion time of the job, then the lateness of job j is $L_j = C_j - d_j$. We let $L_{\max}(\sigma) = \max_j \{L_j\}$.

4.1. On-line optimal dispatching rules for both preemption models

Using the results for the static version of the problem, we have the following Lemma. We omit the proof for brevity.

Lemma 4.1. *At the beginning of the schedule, and at every job completion time, the earliest due date rule is on-line optimal: sequence the available jobs in non-decreasing order of d_j .*

Let $(n+1)$ be the job arriving at time r_{n+1} , when there were n jobs available, with job k on the machine. Assume

Table 2. Heuristic performance for the static non-preemptive total flow time problem

N	Preempt-repeat		C/HP		Combined algorithm	
	Error(%)	Best	Error(%)	Best	Error(%)	Tie ^a
20	2.2350	30	3.7236	3	2.1638	17
40	4.3234	43	6.1124	7	4.0254	0
60	6.6790	29	6.7594	21	5.3096	0
80	7.7358	8	4.8796	42	4.7044	0
100	4.8980	4	2.1802	46	2.1154	0
Average	5.1742		4.7310		3.6637	

^a Number of times the preempt-repeat and C/HP algorithms gave the same schedule

that jobs are labelled such that $d_j \leq d_{j+1}$, $1 \leq j \leq n$. Let B_k and A_k be two subsets of the jobs such that $j \in B_k$ if $d_j < d_k$, and $j \in A_k$ if $d_j > d_k$, for $j \leq n+1$ and $j \neq k$: if k gets preempted, the on-line optimal EDD schedule has jobs in B_k scheduled before k , and jobs in A_k after k . Let σ and σ' be the non-preemption and preemption schedules for the $n+1$ jobs, C_j , L_j and C'_j , L'_j the completion time and lateness of job j in those schedules, and $L(A_k)$, $L(B_k)$, $L'(A_k)$, and $L'(B_k)$ the maximum lateness of jobs in A_k and B_k under σ and σ' , respectively.

Lemma 4.2. *At time r_{n+1} , if the maximum lateness of the non-preemption schedule including job $(n+1)$ is the lateness of either job k or some job $j \in A_k$, it is on-line optimal not to preempt job k .*

Proof: Consider σ and σ' at the arrival of $(n+1)$:

$$\sigma: (k, B_k, A_k);$$

$$\sigma': (\underline{k}, B_k, k', A_k).$$

As $C'_k > C_k$ and $C'_j \geq C_j \forall j \in A_k$, and $C'_j < C_j \forall j \in B_k$ under both preemption models,

$$L_{\max}(\sigma') = \max\{L'_k, L'(A_k)\} \geq \max\{L_k, L(A_k)\} = L_{\max}(\sigma). \quad \blacksquare$$

Let δ_i be the time delay associated with preemption at time r_i , given by:

$$\begin{aligned} \delta_i &= \min\{r_i - X_k, s_k\} && \text{for the preempt-setup model;} \\ \delta_i &= \alpha_k(r_i - X_k) && \text{for the preempt-startup model.} \end{aligned}$$

Theorem 4.3. *A necessary condition for preemption to be on-line optimal at the arrival of some job h is that d_h be the smallest due date of any job available and not completed at time r_h .*

Proof: At time r_h , let there be n jobs available and not completed, labelled such that $d_j \leq d_{j+1}$ for $1 \leq j \leq n-1$. If more than one job arrives at time r_h , let h be the arriving job with smallest due date. Let k be on the machine at time r_h , and $d_h > d_1$. Let $B_k(i) = \{j | j \neq k, d_j \leq d_k, r_j \leq r_i\}$, $A_k(i) = \{j | d_j > d_k, r_j \leq r_i\}$, and let σ_j and σ'_j be the non-preemption and preemption schedules at time r_j . We can write

$$\sigma_h: (k, B_k(h), A_k(h));$$

$$\sigma'_h: (\underline{k}, B_k(h), k', A_k(h)).$$

By Lemma 4.2, $L_{\max}(\sigma_h) = L_h(u)$ for some $u \in B_k(h)$, where $L_i(j)$ and $L'_i(j)$ denote the lateness of job j in σ_i and σ'_i respectively. Let v be the last job to arrive immediately before h . Let $L_{\max}(\sigma_v) = L_v(y)$. Because the lateness of all jobs j with $d_j \leq d_k$ is greater than or equal to the lateness of k in any non-preemption schedule, $y \neq k$. We now show that if $y \neq u$, then $y \in B_u(h)$.

Assume first that $r_u \leq r_v$. Because $L_v(y) \geq L_v(u)$ but $L_h(u) \geq L_h(y)$, the completion time of u increased by more than that of y at time r_h so that at least one job j with $r_j = r_h$ was scheduled between y and u in σ_h : hence if $y \neq u$, then $y \in B_u(h)$.

Assume now that $r_u > r_v$ (i.e., $r_u = r_h$). Of all jobs j with $r_j \leq r_v$, let w be the job with largest due date subject to $d_w \leq d_u$: by definition, in σ_h , any job j scheduled between w and u must have $r_j = r_h$. Assume that $y \in A_k(v)$. By assumption, $L_v(y) > L_v(w)$, which implies

$$\begin{aligned} X_k + \sum_{i \in B_y(v)} t_i + t_y - d_y &> X_k + t_k + \sum_{i \in B_w(v)} t_i + t_w - d_w \\ \Rightarrow d_y - d_w &< \sum_{i \in \{A_w(v) \cap B_k(v)\}} t_i + \sum_{i \in \{A_k(v) \cap B_y(v)\}} t_i + t_y. \end{aligned} \quad (7)$$

However, by assumption, $L_h(u) \geq L_h(y)$, which implies

$$\begin{aligned} X_k + t_k + \sum_{i \in B_u(h)} t_i + t_u - d_u &\geq X_k + \sum_{i \in B_y(h)} t_i + t_y - d_y \\ \Rightarrow d_y - d_u &\geq \sum_{i \in \{A_u(h) \cap B_k(h)\}} t_i + \sum_{i \in \{A_k(h) \cap B_y(h)\}} t_i + t_y, \end{aligned} \quad (8)$$

which contradicts (7) because $d_y - d_w \geq d_y - d_u$.

Assume now that $y \in B_k(v)$ but $y \in A_u(h)$. Using the same argument, we can show that

$$d_y - d_w < \sum_{i \in \{A_w(v) \cap B_y(v)\}} t_i + t_y, \quad (9)$$

$$d_y - d_u \geq \sum_{i \in \{A_u(h) \cap B_y(h)\}} t_i + t_y; \quad (10)$$

which also leads to a contradiction. Thus if $y \neq u$, then $y \in B_u(h)$.

As preemption was not desirable at time r_v , we prove that it will not be desirable at time r_h by showing that

$$L_{\max}(\sigma_h) - L_{\max}(\sigma_v) \leq L_{\max}(\sigma'_h) - L_{\max}(\sigma'_v). \quad (11)$$

Assume that $r_u < r_h$. Let $\Delta_j = L_h(j) - L_v(j)$. Then $\Delta_y \leq \Delta_u \leq \Delta_j \forall j \in A_u(v)$ as $d_y \leq d_u \leq d_j$ and the completion time of y and u will not be affected by jobs arriving at time r_h with due date greater than d_u . Preemption at time r_v will cause the lateness of y , u , and all jobs $j \in B_k(v)$ to decrease by $\lambda(v)$, and the lateness of all jobs in $A_k(v)$ will increase by δ_v and the lateness of k will increase by $\mu_k(v)$. It is easy to verify that

$$\lambda(v) = X_k + t_k - r_v > X_k + t_k - r_h = \lambda(h); \quad (12)$$

$$\delta_v \leq \delta_h; \quad (13)$$

$$\mu_k(v) = \sum_{i \in B_k(v)} t_i + \delta_v \leq \sum_{i \in B_k(h)} t_i + \delta_h = \mu_k(h). \quad (14)$$

Furthermore, $\mu_k(h) - \mu_k(v) \geq \Delta_u + \delta_h - \delta_v$. Hence

$$\begin{aligned} L_{\max}(\sigma_h) - L_{\max}(\sigma_v) &\leq \Delta_u \leq \Delta_u + \delta_h - \delta_v \\ &\leq L_{\max}(\sigma'_h) - L_{\max}(\sigma'_v). \end{aligned} \quad (15)$$

Assume now that $r_u = r_h$. It can be shown that $L_{\max}(\sigma_h) - L_{\max}(\sigma_v)$ equals

$$L_h(u) - L_v(y) = \sum_{i \in B'_u(h)} t_i + t_u + d_y - d_u, \quad (16)$$

where $B'_u(h) = \{j \mid j \neq u, d_j \leq d_u, r_j = r_h\}$. However, $\forall j \in \{k \cup A_k(v)\}$,

$$L_h(j) - L_v(j) = \sum_{i \in B'_j(h)} t_i \geq \sum_{i \in B'_u(h)} t_i + t_u \geq L_h(u) - L_v(y), \quad (17)$$

because $d_j > d_u \forall j \in \{k \cup A_k(v)\}$ and $d_y \leq d_u$. Hence the increase in maximum lateness in going from σ_v to σ_h is no greater than the increase in the lateness of all jobs $j \in \{k \cup A_k(v)\}$. Because (12), (13) and (14) still hold, so does (11), and the proof is complete. ■

It follows that we will only preempt in favor of the available job with smallest due date. We now describe conditions under which such preemption would be worthwhile. As before, let k denote the job being processed at the arrival of job $(n+1)$, σ and σ' the non-preemption and preemption schedules respectively, L_j and L'_j the lateness of job j in those schedules, and $L(A_k)$, $L(B_k)$, $L'(A_k)$ and $L'(B_k)$ the maximum lateness of jobs in A_k and B_k under those schedules, respectively. Assume that job k started at time X_k , with $r_{n+1} < X_k + t_k$.

Theorem 4.4. *Given the arrival of job $(n+1)$ at r_{n+1} , where $(n+1) \in B_k$, and unprocessed jobs $A_k \cup B_k \cup k$, it is on-line optimal to preempt if and only if*

$$\delta_{n+1} + \max \left\{ L_k + \sum_{j \in B_k} t_j, L(A_k) \right\} < L(B_k). \quad (18)$$

Proof: Let σ and σ' be the following non-preemptive and preemptive schedules at time r_{n+1} :

$$\begin{aligned} \sigma: & (k, B_k, A_k); \\ \sigma': & (\underline{k}, B_k, k', A_k). \end{aligned}$$

Note that it follows from Lemma 4.1 and Theorem 4.3 that if job $(n+1)$ does preempt job k , then it will begin processing at time r_{n+1} . It follows that

$$L_{\max}(\sigma) = \max\{L_k, L(A_k), L(B_k)\}; \quad (19)$$

$$L_{\max}(\sigma') = \max \left\{ L_k + \sum_{j \in B_k} t_j + \delta_{n+1}, L(A_k) + \delta_{n+1}, L(B_k) - t_k + r_{n+1} \right\}. \quad (20)$$

For any $\alpha > 0$, in the preempt-startup model, or any non-zero setup time in the preempt-setup model, the first and second terms in (20) are larger than the first and second terms, respectively, in (19), but the third term in (20) is smaller than in (19). Hence we need only compare the

third term in (19) with the first and second terms in (20); preemption is worthwhile if and only if

$$\max \left\{ L_k + \sum_{j \in B_k} t_j + \delta_{n+1}, L(A_k) + \delta_{n+1} \right\} < L(B_k);$$

hence the result. ■

Again the result is intuitive: preemption is worthwhile only if the increase in the lateness of jobs in A_k and k , i.e., the only jobs whose lateness will increase because of the preemption, is no more than the maximum lateness of jobs in B_k in the non-preemption option.

From this result, an on-line optimal dispatching algorithm can be described to solve the dynamic maximum lateness problem under both preemption models, as was done for the total weighed flow time case in the previous section. For brevity we do not present this description.

4.2. Preempt-repeat heuristic for the non-preemptive static problem

The on-line dispatching rules developed in the Section 4.1 will generate optimal solutions for a special case of the non-preemptive static maximum lateness problem similar to that of the total weighted flow time problem. Specifically, assume that the preempt-repeat schedule obtained by these dispatching rules is such that Isolated Clusters, as defined by conditions C1 to C3 (where now $[k]_j$ stands for the job with k th smallest due date in IC_j), can be formed, and C4 holds. Then the jobs within each Isolated Cluster will be optimally scheduled in earliest due date order provided that the job with smallest due date in the Isolated Cluster arrives early enough to preempt any job of the Isolated Cluster that has started processing. This condition, which could be denoted C'5 in the description of these Isolated Clusters, is equivalent to (18) in Theorem 4.4; the proof is similar to that of Theorem 3.6, and is omitted.

We now demonstrate how poorly the on-line preempt-repeat algorithm can perform for the static non-preemptive maximum lateness problem. Because the optimal value for an instance of the maximum lateness problem can be zero or negative, we use the worst-case performance difference instead of the ratio. Assume that the jobs are numbered such that $d_1 \leq \dots \leq d_N$.

Job j	1	2	3
r_j	0	$2P - 1$	$2P$
p_j	$6P$	$2P - 1$	$2P$
d_j	$10P - 1$	$8P - 1$	$10P - 1$

An optimal schedule is $\{1, 2, 3\}$, producing competition times of $6P$, $8P - 1$ and $10P - 1$ for the three jobs respectively. The maximum lateness of this schedule is 0. The preempt-repeat algorithm preempts job 1 in favor of job 2 at time $2P - 1$, then schedules job 2 to finish at time

$4P - 2$, and job 3 to finish at time $6P - 2$. Finally, job 1 is scheduled to finish at time $12P - 2$, which is $2P - 1$ units of time late. We conjecture that the worst case difference is $r_{\max}$, the largest ready time, as is suggested by this example.

5. Continuous improvement: reducing preemption time delays

The dispatching rules developed in Sections 3 and 4 provide an interesting alternative to other on-line algorithms that do not allow preemption because they give the scheduler the added flexibility to reschedule the jobs in the system, including the job currently on the machine, each time a new job arrives. However, as suggested by many applications, this added flexibility is not necessarily obtained without some kind of 'penalty': we modeled this 'penalty' as a time delay incurred whenever a job is preempted. In particular, preemption might not be an attractive option if the preemption delay corresponds to repeating the job completely. In contrast, if no time delay is associated with preemption, such as in the preempt-resume mode, preemption becomes very desirable because it yields optimal solutions in polynomial time for the objectives considered in this paper (excluding the total weighted flow time problem). To take full advantage of preemption, managers could thus decide to invest in continuous manufacturing improvement, which could take the form of reducing the delays associated with preemption. Porteus [13, 14] has shown the importance of reducing setup times and costs in modern production systems.

The purpose of this section is to investigate the effects of continuous improvement on the solutions obtained by our preemption algorithms. In our analysis, we model continuous improvement as a reduction of the size of the setup times, and of the constants α_j , which would tend to make preemption more desirable. We report the effects of such continuous improvement resulting from computational experiments for the performance measures studied in this paper.

It is important to note that a decrease in α_j or s_j will always result in a better or equally good solution, assuming that r_j and p_j remain unchanged $\forall j$, provided that an algorithm can yield an optimal solution to the problem for any value of α_j and s_j . However, the algorithms suggested here cannot guarantee optimality, so that even if the α_j or s_j are reduced, the solution obtained by the heuristics might be worse than with larger values of α_j or s_j because of heuristic error. A reduction in α_j or s_j is likely to make preemption more desirable for the heuristic. The worst-case performance of the heuristics suggests that algorithms might perform poorly when they choose preemption but the optimal solution does not. From these two observations, it is reasonable to

expect that the heuristic solution might occasionally get worse as a result of the reduction of α_j or s_j : this indeed happened during the computational experiments.

For the experiments involving the preempt-startup heuristics, we randomly generated problems where ready times r_j were uniformly distributed over the integers $1, \dots, 2000$, processing times p_j were uniformly distributed over the integers $1, \dots, 50$, and due dates were uniformly distributed over the integers $r_j + p_j, \dots, r_j + p_j + 200$. Given that all jobs arrive within 2000 time units, we modelled the traffic density of the shop by using different values of N , the number of jobs: we let $N = 20, 40, 60, 80$, and 100. For each N we generated 10 problems, and for each problem we let $\alpha_j = \alpha = 0.9, 0.7, 0.5, 0.3$, and 0.1.

For the experiments involving the preempt-setup heuristics, we randomly generated problems where ready times r_j were uniformly distributed over the integers $1, \dots, 3000$, processing times p_j were uniformly distributed over the integers $1, \dots, 50$, setup times $s_j = \beta p_j$ and due dates were uniformly distributed over the integers $r_j + p_j, \dots, r_j + p_j + 200$. Again, because all jobs arrive within 3000 time units, we modelled the traffic density of the shop by using different values of N , the number of jobs: we let $N = 20, 40$, and 60. For each N , we generated 10 problems for the total flow time objective, and 25 problems for the maximum lateness objective; a decrease in setup times was modelled by decreasing the value of β : $\beta = 3, 2, 0.9, 0.7, 0.5, 0.3$ and 0.1.

Heuristic solutions were tested against solutions obtained by the respective optimal preempt-resume algorithm (i.e., where $\alpha = 0$ or $\beta = 0$), a lower bound on the optimal solution for both the total flow time and maximum lateness objectives, and also the ultimate goal to achieve in the continuous improvement effort: in comparing the heuristic solution with the lower bound we learn about the effectiveness of the heuristic, and about how fast we converge on this goal.

Table 3 gives the average percentage heuristic error above the lower bound for the total flow time preempt-startup model for different values of N and α . It also gives the average number of times that preemption was considered by the heuristic, 'Try', i.e., when $t_{n+1} < t'_{[k]}$, and the average number of times the heuristic did preempt, 'Preempt', i.e., based on Theorem 3.4. As reported below, a decrease in α leads to an approximately linear reduction in the average percentage heuristic error: this is also true of each individual instance although the rate of the improvement varied significantly, even between instances with the same number of jobs. The heuristic preempted more often with smaller α : as shorter jobs were replacing longer ones on the machine, preemption was attempted less often.

Table 4 gives similar results for the total flow time preempt-setup problem. As the lower bound solution was obtained by assuming $\beta = 0$, heuristic errors were obvi-

Table 3. Average percentage heuristic errors, preempt-startup model, total flow time problem

N		α				
		0.9	0.7	0.5	0.3	0.1
20	Error (%)	2.636	2.147	1.619	1.089	0.450
	Try	3	2.8	2.6	2.1	1.9
	Preempt	1.2	1.2	1.1	1.1	1.5
40	Error (%)	3.765	3.543	2.853	2.273	0.956
	Try	12.3	11.1	9.7	8.8	6.9
	Preempt	2.8	3	3.2	3.9	5.3
60	Error(%)	5.036	5.232	4.595	3.150	1.679
	Try	25.4	23.3	21.1	17.6	14.5
	Preempt	4.9	6.5	7.5	8.9	12.1
80	Error (%)	5.799	5.194	4.746	3.932	2.342
	Try	44.4	39.5	34.3	29.1	25
	Preempt	7	8.1	9.2	11.3	15.9
100	Error (%)	4.097	3.685	3.806	3.075	1.980
	Try	53.5	47.3	40.9	35.6	30.9
	Preempt	5.2	5.6	8.1	11.1	18

ously very large for large β . A decrease in β resulted in an approximately exponential reduction in the average percentage heuristic error.

Table 5 also gives the average percentage heuristic error above a lower bound for the total flow time preempt-setup model, but now the lower bound is a function of β . Instead of assuming that $\beta = 0$ and using the solution from the optimal preempt-resume algorithm on the processing times p_j as a lower bound, we use the same algorithm, but on $t_j = s_j + p_j = (1 + \beta)p_j$ as a lower bound. The results suggest that as β decreases, either the effectiveness of the heuristic improves or the quality of the bound improves, which would imply that the heuristic is close to optimal for all β .

For the maximum lateness problem, the effect of reducing α or β on the difference between the maximum lateness of the heuristic and the lower bound was studied for different values of N . For the preempt-startup model, preemption was not attempted very often by the heuristic (preemption is attempted if the due date of the arriving job is the smallest due date of all available jobs), and did

not occur very often (based on Theorem 4.4), as shown in Table 6, which also shows in how many instances the maximum lateness decreased as α decreased for each N . For the preempt-setup model, although the preemption attempts by the algorithm seem comparable with the preempt-startup model, preemption occurs much more often, as shown in Table 7, but the effect on the maximum lateness of reducing β is much less predictable. This suggests that if the objective is to minimize the maximum lateness, the heuristics are not as effective, as decreases in α and β resulted in no change or increases in the objective value more often than for the total flow time problem. Moreover, results show that the direction and magnitude of the change in the objective value following a decrease in α and especially β are much more difficult to predict for the maximum lateness problem: this difficulty is likely to persist even if an optimal algorithm for general α_j or s_j is developed.

From these computational experiments, we believe that the issue of investing in continuous reduction in job setups or work repetition fractions (α) definitely deserves further investigation, especially if the objective is to minimize total flow time.

6. Conclusions

In this paper we have studied two generalized preemption models, for which we have developed on-line optimal dispatching rules for the single-machine dynamic scheduling problems of minimizing total (weighted) flow time and minimizing maximum lateness. These models were motivated by the fact that allowing preemption can yield important flow time benefits, even if some time penalty must be incurred for preempted jobs. In an on-line environment, if preemption is allowed, decision makers can keep the machine busy as long as there are jobs available, knowing that any job can be interrupted if a more urgent one arrives. Our dispatching rules thus provide an interesting alternative to other on-line algorithms that do not allow preemption, because they give the scheduler the

Table 4. Average percentage heuristic errors, preempt-setup model, total flow time problem

		β						
		3	2	0.9	0.7	0.5	0.3	0.1
20	Error(%)	532.51	301.27	119.37	90.69	63.39	36.70	10.79
	Try	6	4.6	2.4	2.4	2.1	1.6	1.2
	Preempt	0.1	0.2	0.6	0.5	0.4	0.7	1.2
40	Error(%)	1928.3	693.05	168.86	123.74	83.01	43.81	12.98
	Try	20.3	15.7	9	8.4	7.9	5.5	4.4
	Preempt	0.1	0.5	0.9	1.4	1.1	2.1	3.1
60	Error(%)	3991.5	1906.1	312.91	181.99	108.4	56.16	15.65
	Try	28.9	25.7	21.8	18.3	15.1	12.4	10.1
	Preempt	0	0.2	1.3	2.4	2.3	3.3	6.6

Table 5. Average percentage heuristic errors, preempt-setup model, total flow time problem

		β						
		3	2	0.9	0.7	0.5	0.3	0.1
N								
20	Error(%)	13.980	8.806	4.844	4.273	3.424	2.207	0.019
40	Error(%)	45.930	24.781	7.089	5.867	4.719	2.290	0.678
60	Error(%)	56.860	45.174	18.292	11.159	8.498	5.141	1.088

Table 6. Aggregate results, preempt-startup model, maximum lateness problem

N	Try (average)	Preempt (average)	Effect on $L_{\max}$ as α decreases (10 problems for each N)			
			No change (number)	Decrease (number)	Increase (number)	Mixed (number)
20	2	0.32	8	2	0	0
40	5.9	0.66	5	5	0	0
60	13.58	1.02	6	3	1	0
80	10.48	0.88	8	0	1	1
100	6.6	1.88	3	3	2	2

added flexibility to reschedule the jobs in the system, including the job currently on the machine, each time a new job arrives.

It was noted that if s_j is close to zero for all jobs j in the preempt-setup model, or if α_j is close to zero in the preempt-startup model, the resulting dispatching rules are closely related to the well-known preempt-resume mode. Conversely, if p_j is very small compared with s_j in preempt-setup and α_j is close to 1 for each job j in preempt-startup, both models converge to the preempt-repeat mode, where preempted jobs must be totally repeated. A preempt-repeat heuristic based on this insight was suggested for the non-preemptive static problems listed above, where all job ready times are known and job preemption is not permitted. We have provided conditions under which the preempt-repeat algorithm was optimal for the static non-preemptive problem and constructed examples showing poor performance of the heuristic.

In comparing the preempt-repeat algorithm with the preempt-resume mode we note that for a given situation (job $[k]$ on the machine, n jobs in the system, job $(n+1)$ arriving) if preemption is not desirable for the preempt-setup or preempt-startup algorithm because $\rho_{n+1} > \rho_{[k]}$, for the flow time problem, or $d_{n+1} > d_{[k]}$ for the lateness problem, then it is not desirable to preempt even in the

preempt-resume mode. Furthermore, although in a preempt-resume environment, preemption in favor of the arriving job is optimal, this remains true for our more general preemption models for the maximum lateness problem, but this is not necessarily the case for the flow time problem, an interesting and perhaps counter-intuitive result.

Interesting issues resulting from our work include the study of preemption-startup assumptions for a static problem (i.e., where look-ahead information is available). A comparative study between the dynamic and static preemption models might reveal interesting insights concerning the value of look-ahead information if such information was known in probability, thus suggesting stochastic modelling of the problems. The derivation of on-line optimal dispatching rules for other objective functions, or more complex machine environments, is also worth consideration. However, such problems should possess certain characteristics. For example, whereas the dynamic problem may be hard, the static version should be polynomially solvable; otherwise, once a job is preempted, it becomes a difficult problem to decide which job should next begin processing. Moreover, it should be pointed out that it is possible that such a decision cannot be considered separately from whether or not to preempt in the first place. An example of such a

Table 7. Aggregate results, preempt-setup model, maximum lateness problem

N	Try (average)	Preempt (average)	Effect on $L_{\max}$ as β decreases (25 problems for each N)			
			No change (number)	Decrease (number)	Increase (number)	Mixed (number)
20	2.011	1.594	7	8	1	9
40	4.543	3.589	6	4	0	15
60	7.246	5.309	2	1	0	22

situation occurs for the problem of minimizing total tardiness, which even in its static form, has been shown to be NP-hard in the ordinary sense [15]. It would be possible to describe rules for when to preempt by making a restrictive assumption about the next job to be scheduled (for example, that it is always the most recently arrived job). However, such rules may not be on-line optimal.

The possibility of dynamically increasing the weights of jobs as a function of the time spent in the system also seems an attractive heuristic idea: more precisely, if an infinite horizon is assumed, such a model could prevent long jobs with small weights from continuously being preempted. Our worst-case analysis suggests that this could improve heuristic performance.

We hope that this paper will motivate more research on dynamic models when rescheduling is allowed, and on generalized models of preemption.

Acknowledgements

We wish to acknowledge the helpful insight provided by David Mossman during the early stage of this research, as well as the invaluable suggestions of Jeff Sidney and Kathie Cameron which improved both the presentation and the content of this paper significantly. This research was partly supported by NSERC grants No. A4124 and No. OGP0105575.

References

- [1] Baker, K.R. (1974) *Introduction to Sequencing and Scheduling*, John Wiley and Sons.
- [2] Lenstra, J.K., Rinnooy Kan, A.H.G. and Brucker, P. (1977) Complexity of machine scheduling problems. *Annals of Discrete Mathematics*, 1, 75–90.
- [3] Smith, W.E. (1956) Various optimizers for single-state production. *Naval Research Logistics Quarterly*, 3 (1), 59–66.
- [4] Jackson, J.K. (1955) Scheduling a production line to minimize maximum tardiness. Research Report 43, Management Science Research Project, University of California, Los Angeles.
- [5] Bianco, L. and Ricciardelli, S. (1982) Scheduling of a single machine to minimize total weighted completion time subject to release dates. *Naval Research Logistics Quarterly*, 29 (1), 151–167.
- [6] Chandra, R. (1979) On $n/1/\bar{F}$ dynamic deterministic problems. *Naval Research Logistics Quarterly* 26, 537–544.
- [7] Deogun, J.S. (1983) On scheduling with ready times to minimize mean flow time. *Computing Journal*, 26 (4), 320–338.
- [8] Dessouky, M.I. and Deogun, J.S. (1981) Sequencing jobs with unequal ready times to minimize mean flow time. *SIAM Journal of Computing*, 10 (1), 192–202.
- [9] Hariri, A.M.A. and Potts, C.N. (1983) An algorithm for single machine sequencing with release dates to minimize total weighted completion time. *Discrete Applied Mathematics*, 5, 99–109.
- [10] Beshara, S.B. and Magazine, M.J. (1981) Myopic heuristics for single machine scheduling problems. *International Journal of Production Research*, 19 (1), 85–95.
- [11] Posner, M.E. (1986) A sequencing problem with release dates and clustered jobs. *Management Science*, 32 (6), 731–738.
- [12] Conway, R.W., Maxwell, W.L. and Miller, L.W. (1967) *Theory of Scheduling*, Addison-Wesley, Reading, MA.
- [13] Porteus, E.L. (1985) Investing in reduced setups in the EOQ model. *Management Science*, 31 (8), 998–1010.
- [14] Porteus, E.L. (1986) Optimal lot sizing: process quality improvement and setup cost reduction. *Operations Research*, 34 (1), 137–144.
- [15] Du, J. and Leung, J.Y.-T. (1990) Minimizing total tardiness on one machine is NP-hard *Mathematics of Operations Research*, 15, 483–495.

Biographies

François M. Julien is Assistant Professor in the Management section of the Faculty of Administration at the University of Ottawa, Ontario, Canada. He received his Ph.D. in management science from the University of Waterloo, Ontario, in 1991. His current research interests are in production scheduling, operations management, real-time control and dynamic scheduling of buses within an urban public transportation network.

Michael Magazine is Professor and Ohio Eminent Scholar in the Department of Quantitative Analysis and Operations Management at the University of Cincinnati. He was previously on the faculty of the University of Waterloo and was the Area Editor for *Operations Research* in Operations, Planning and Scheduling. His interests are in agile manufacturing and supply chain management.

Nicholas G. Hall is Professor of Management Science and Operations Management at The Ohio State University. He holds a Ph.D. in management science from the University of California, Berkeley (1986). His research interests are in manufacturing and scheduling problems, as well as in applications of operations research. He serves as Associate Editor in manufacturing and scheduling areas of the journals *Operations Research*, *Management Science* and *IIE Transactions*.